Local time stepping and custom kernels: design and phases

Items 7 and 8 of the Nitsche profiling plan: the idea, how each fits the code at `53ba400`, and a phased build with a test at every phase · revision 1, 17 Oct 2026

Revision 1. First version, written for Chunhui Zhao to read before any implementation. It is a design only, and no code is changed. New numbers come from two mesh-statistics scripts (§7.6). Everything else cites a code line at `53ba400`, the profiling report (`nitsche-profiling-plan-rev1.pdf`, cited as "profiling §n"), a repository document or a paper. Inferences are labelled as such.

1. Summary

Item 7 is clustered local time stepping (LTS) with rate 2: each tet steps at the largest $\Delta t_0 2^c$ its own size allows, and only the tets of cluster c are updated at that rate. **Item 8** replaces the two assembled sparse products Ku and $K_\eta v$, which take 84 to 91% of a step (profiling §4.2), with a kernel that works element by element and face by face and reads far fewer bytes. The recommendations are these.

1. **Do item 8 before item 7.** Item 8 does not change results, because every phase checks Ku against the assembled matrix to round-off, so the existing golds stay valid. It also builds the per-element and per-face loops that LTS needs: an LTS step applies the operator to one cluster's elements only, which an assembled `HypreParMatrix` cannot do cheaply.
2. **Do item 8 in two steps.** First, store K and K_η as symmetric dense blocks and apply both in one sweep (8a). This reads 2.8 times fewer bytes than today's two CSR products at $p = 1$ and $p = 2$, so it should run the products about 2.1 to 2.7 times faster (inferred from bytes and the measured dense-block efficiency, §4.2). Second, a matrix-free kernel built from reference matrices and per-element geometry (8b). It reads 13 to 100 times fewer bytes, but its speed is set by how fast small loops run. At 1 to 5 GFLOP/s per core the model gives 4 to 16 times at $p = 1$ and 0.9 to 6 times at $p = 2$, so a prototype must measure it first (§4.2).
3. **For item 7, use a multirate leapfrog in physical time, not the Diaz–Grote leapfrog LTS.** The Diaz–Grote scheme computes the refined region at substeps that are not physical times, so the friction closure cannot run there. On both 200 m meshes the smallest tets sit within two element layers of the fault, so the refined region always reaches the fault (§3.3). In the recommended scheme every cluster takes ordinary leapfrog steps in physical time, and the fault runs its existing algorithm at its own cluster's step, as in SeisSol (§3.4).
4. **Gains of item 7.** Ideal LTS cuts element updates 4.54 times on TPV205 200 m and 3.77 times on TPV104 200 m with SeisSol's cluster rules. The recommended scheme computes each face between two clusters once per side, so a block count gives 3.87 and 3.24 times at $p = 1$ (§3.1). Parallel runs will realise less: the velocity-stress path's own LTS lost 40% of its wall time to exposed MPI wait (§3.5). Better meshes (item 2) take part of the gain.

On the 1000 m mesh the gmsh optimiser alone gives 2.0 times, LTS alone 3.76 times, and both 5.1 times, in ideal element updates (§3.1).

Item	What changes	Expected gain	Results	Effort and main risk
8a	Symmetric dense blocks of K, K_η; one fused sweep	2.1 to 2.7× on the products, 1.8 to 2.2× per step (inferred)	Same to round-off	Medium; low risk, since the entries are the assembled ones
8b	Matrix-free fused kernel on affine tets	4 to 16× at $p = 1$, 0.9 to 6× at $p = 2$ on the products (model)	Same to round-off	Large; the flop rate of small kernels is unknown
7	Clustered multirate leapfrog; fault at its cluster's step	3.2 to 3.9× serial by block count at $p = 1$ on the 200 m meshes; parallel share unknown	New golds	Large; stability with damping and the fault is not proven

Both items touch the same two places: the operator apply in `Acceleration_` and the time loop around it. Item 8 changes the apply only. Item 7 changes the loop and needs the apply restricted to one cluster. Kelvin–Voigt damping enters both: item 8 fuses $K_\eta v$ into the same sweep, and item 7 must keep the damped stability limit in every cluster (§3.4, step 5).

2. Today's step, and where items 7 and 8 plug in

The driver advances the operator with MFEM's `CentralDifferenceSolver`, which is `NewmarkSolver(0, 0.5)` (ode.hpp:927), one `ode.Step` per step (nitsche_dyn_driver.cpp:1441-1444). The step is one global $\Delta t = \mathrm{cfl}\, C(p) \min_e h_e/(p^2 c_{p,e})$ with $h_e = 6V_e/A_e$ (nitsche_cfl.hpp:69, called at driver:991). Figure 1 shows one step and the two plug-in points; the table after it lists the functions on the path.

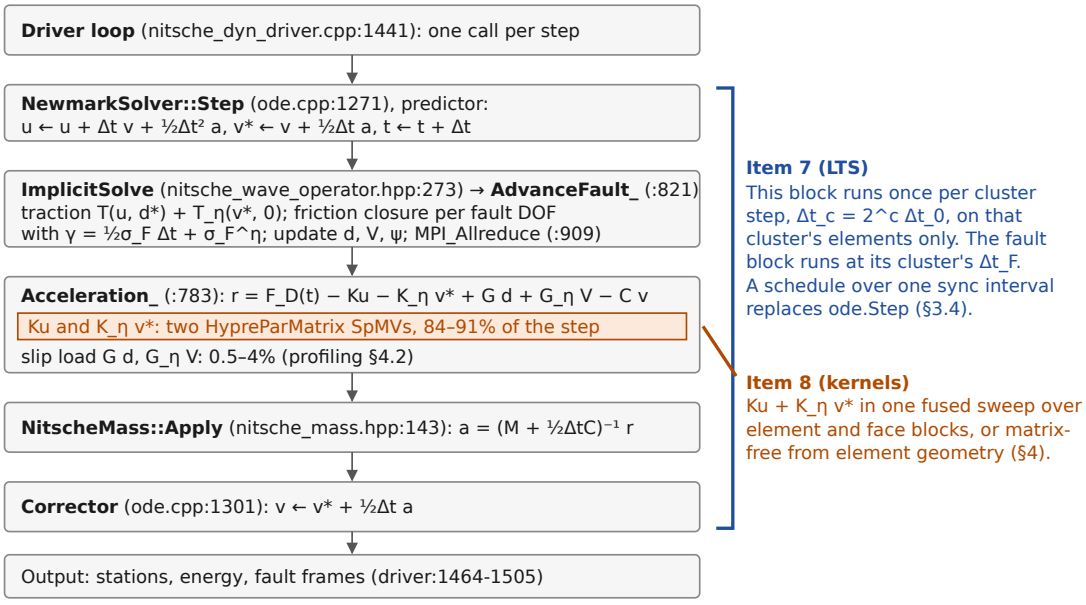


Figure 1. One central-difference step at `53ba400`. Shares are of a serial $p = 1$ step (profiling §4.2). Item 8 replaces the orange line; item 7 replaces the loop around the blue bracket.

Function (link)	Inputs → outputs	What it does	Item
NewmarkSolver::Step	$u, v, t \to$ new u, v, t	Predictor, one `ImplicitSolve`, corrector; the first call computes a^0 with `Mult`.	7 (replaced)
ImplicitSolve	$u^{n+1}, v^* \to a^{n+1}$	Checks that the solver's Δt matches `SetTimeStep` (:276-289), advances the fault, computes the acceleration.	7
AdvanceFault_	$u^{n+1}, v^*, t \to d, V, \psi$	Half-step slip, trial traction, closure with γ (ClosureGamma :697), state update, one `MPI_Allreduce`.	7 (at Δt_F)
Acceleration_	$u, v, d \to a$	Forms the residual of Figure 1 and applies the mass inverse.	8, 7 (per cluster)
ApplyStiffness	$u \to Ku$	One `HypreParMatrix::Mult` with its own halo exchange; called for K and again for K_η.	8 (replaced)
AssembleStiffness	$\lambda, \mu \to K$	Volume integrator plus the combined SIPG face integrator on all interior faces (fault faces included) and Dirichlet faces; `KeepNbrBlock(true)` and `Assemble(0)` make K partition-independent (:25-34, :80-85).	8 (source of blocks)
AssembleFaceMatrix	face $\to 2 \times 2$ block matrix	Consistency, symmetry and penalty in one pass, quadrature order $2p+1$, exact symmetrisation (:183-194).	8, 7 (split by side)
SetKelvinVoigt	$\bar{\eta} \to K_\eta, \sigma_F^\eta$	Sets $\beta_e = \bar{\eta}\frac{1}{3}(h_e/p)/c_{s,e}$ (:627) and builds a second SIPG operator with moduli $\beta_e\lambda_e, \beta_e\mu_e$ (:656).	8 (fused), 7
NitscheMass::Apply	$r \to a$	Element-local mass inverse, with dense $(\rho M_e \otimes I_3 + \frac{1}{2}\Delta t\, C_e)^{-1}$ on absorbing elements; Δt is fixed at SetTimeStep :241.	7 (per-element Δt)

Two facts matter for both items. First, K treats a fault face as an ordinary interior face; slip enters only through the load Gd (`AssembleSlipOnlyRHS`, traction.inl:618), so a custom K kernel needs no fault logic. Second, the damping term uses the half-step velocity v^*, so it is explicit and lagged by half a step (nitsche_wave_operator.hpp:70-92), while the absorbing term is implicit but element-local.

3. Item 7: clustered local time stepping

3.1 What clustering can recover on the benchmark meshes

The single worst tet sets today's step for the whole mesh (profiling §4.6). Rate-2 clustering gives tet e the step $\Delta t_0 2^{c_e} \le \Delta t_e$. SeisSol adds two rules: face neighbours differ by at most one cluster, and the two tets of a rupture face share a cluster. The repository implements exactly these rules for the velocity-stress path in `BuildLtsClustering`

(dynamic/lts_clustering.hpp:79). The script of §7.6 applies them to the Nitsche length $h_e = 6V/A$; TPV205 and TPV104 are homogeneous, so $\Delta t_e \propto h_e$.

Mesh (tets)	$h_{\min}$**; at the fault**	**Ideal: rules;** $N_c \le 4$**;** $N_c = 2$	**Fault faces in** $c = 0$**;** $c = 1$	**Seam faces**	**Block count** $p = 1$**: trace; full**
TPV205 200 m (1.90 M)	26.2; 39.3 m	**4.54**; 4.24; 1.99	3; 26,639	23.1%	**3.87**; 3.16
TPV104 200 m (2.46 M)	29.7; 49.0 m	**3.77**; 3.62; 1.98	29; 37,891	22.9%	**3.24**; 2.66
TPV205 1000 m (187k)	156.4; 268.0 m	3.76; 3.76; 1.98	17; 1119	22.0%	3.23; 2.66
Same, optimised (154k)	262.5; 262.5 m	2.52; 2.52; 1.88	1136; 0	14.1%	2.30; 2.03

"Ideal" is the ratio of element updates per simulated second, global over clustered, which is what `HarmonicUpdateSpeedup` reports (:96). The wiggle-factor scan adds less than 2%. On TPV205 200 m the seven clusters hold 7975, 275k, 799k, 427k, 244k, 132k and 13k tets. The TPV104 figure agrees with the velocity-stress path's report on the repository's mesh (3.63 raw, about 3.68 at its wiggle factor; `jobs/lts_phase5/tpv104_200m_seissol_expanse/README.md`, item 3); our mesh was regenerated from the same geometry and has 1260 fewer tets.

The last column charges the recommended scheme for its seam faces, the faces between two clusters. Each side of a seam face computes its own half of the face terms at its own rate (§3.4, step 2), so these faces are computed twice. "Trace" stores those half-terms on the face nodes only, and "full" stores them as full element blocks, as the assembled K stores its face blocks today. At $p = 2$ the trace figures are 4.05 and 3.38 on the two 200 m meshes. This is a model of memory traffic, not a timing (§7.6). Three observations shape the design.

- **The smallest tets are near the fault but rarely on it.** Half of the smallest 0.1% lie within two element layers of the fault on both 200 m meshes, and none of the smallest 100 touches it. The fault therefore lands in cluster 1.
- **Most of the gain needs few clusters.** Four clusters give 93 to 96% of the ideal, which matters because each cluster level adds exchange rounds (§3.5).
- **Mesh quality and LTS overlap.** The optimiser raises $h_{\min}$ on the 1000 m mesh 1.68 times and cuts the tets to 154k, which is 2.04 times less work under global stepping (profiling §4.6 calls it "about halved"). The smallest tets are then the fault-adjacent ones, and the LTS ideal falls from 3.76 to 2.52. Both together give 5.1 times the original (4.7 by block count). I infer, without measuring, that the 200 m meshes behave alike, so their LTS figures are upper bounds once item 2 is done.

3.2 What an LTS scheme must do in this code

1. **R1, leapfrog.** With one cluster it must be today's central difference, so that the existing golds are its $N_c = 1$ limit (ode.cpp:1271).
2. **R2, Kelvin–Voigt.** $K_\eta v$ is a full SIPG operator (:656), applied explicitly. Its limit $\omega^2 \Delta t(\Delta t + 2\beta) \le 4$ (KV derivation rev 2.1, §6) must hold in every cluster.

3. **R3, absorbing boundary.** $(M + \frac{1}{2}\Delta t\, C)^{-1}$ is element-local (nitsche_mass.hpp:109-131), so it can take each element's own Δt.
4. **R4, fault.** The closure needs a physical u^{n+1} on both sides of every fault face, at the end of a step whose length enters $\gamma = \frac{1}{2}\sigma_F \Delta t + \sigma_F^{\eta}$ and the check $|V|\Delta t / D_c \leq 2$ (:887-889).
5. **R5, restriction.** A cluster update needs its own part of Ku at a cost proportional to the cluster's size.
6. **R6, parallel.** Ghost values that a cluster update reads must be exchanged at that cluster's instants.

3.3 Two families of explicit LTS

(A) Leapfrog LTS of Diaz and Grote (2009). A 0/1 diagonal matrix P selects the refined region. Once per coarse step the scheme computes $w = (I - P)R_n - A(I - P)z_n$, integrates an auxiliary problem with p leapfrog substeps that apply AP, and sets $z_{n+1} = -z_{n-1} + q_1$ (Grote and Mitkova, arXiv:1205.0654, LTS-LF2(p); §7.2). It conserves a discrete energy, and a stabilised version removes the step sizes where the original fails (Grote, Michel and Sauter 2021). With IP-DG the refined region must overlap its neighbours by two elements to reach the full step, otherwise it gets about 60% of it, and damping is treated implicitly (LTS-LFCN), which needs a block-diagonal damping matrix (same chapter).

The auxiliary variable is $q(\tau) \approx z(t_n + \tau) + z(t_n - \tau)$, the even part of the solution about t_n, with sources at $t_n \pm m\Delta\tau$ (§7.2). The fine substeps are therefore not physical times. This is my reading of the verified algorithm, not a statement from the papers. It rules the scheme out here for three reasons.

- The friction closure (R4) cannot run inside the refined region, because it needs the physical u at each substep.
- So the fault must sit in the coarse level, but the refined region with its two-element overlap contains 9,083 of 26,642 fault faces on TPV205 200 m and 18,221 of 37,920 on TPV104 200 m. If the fault and two layers must stay coarse, the coarse step is set by the global minimum tet on both meshes, and LTS gains nothing.
- K_η is not block-diagonal, so LTS-LFCN does not cover Kelvin–Voigt damping (R2).

The velocity-stress path's method study also rejected leapfrog LTS, for its own reasons: proofs for mass-lumped conforming elements and a demonstrated ceiling of about 4 times (`document/lts_dev/ANALYSIS_lts_method_selection_2026-07-18.md`).

(B) Multirate leapfrog in physical time. Every cluster takes ordinary kick–drift–kick steps of its own length. Every DOF moves at constant velocity between the kicks it receives, and each part of the force kicks at the rate of the element that owns it. This is the nested impulse method of molecular dynamics (r-RESPA; Tuckerman, Berne and Martyna 1992). Lew, Marsden, Ortiz and West (2003) apply the same idea to elastodynamics with one step per element (asynchronous variational integrators): momentum and a multisymplectic structure are conserved exactly, and energy behaves very well in tests. Every cluster is at a physical time at its own instants, so the fault runs its existing algorithm at its cluster's step (R4), as SeisSol does for rupture faces. The price is that no energy proof exists, and multiple-time-step methods have resonance instabilities at certain step ratios (Biesiadecki

and Skeel 1993; García-Archilla, Sanz-Serna and Skeel 1998). Phase P1 therefore tests stability directly.

> **Recommendation.** Use family (B), with the operator split by element owner (§3.4, step 2), each fault kept inside one cluster as in SeisSol, and the stability tests of P1 as the gate.

3.4 The recommended scheme, step by step

Step 1: clusters. Compute Δt_e with the expression of `ComputeMaxDt` (nitsche_cfl.hpp:69-91) and call `BuildLtsClustering`. It returns c_e, Δt_0 and the cluster count, and asserts $\Delta t_0 2^{c_e} \le \Delta t_e$ for every element (lts_clustering.cpp:249-259). P3 starts with the whole fault in one cluster, which costs about 6% of the ideal on the 200 m meshes (4.26 instead of 4.54 on TPV205).

> **Why step 2.** The obvious split gives each face term to the finer of its two elements. But a SIPG face term on its own is indefinite: a field with a small jump and a large normal stress makes it negative. The fine cluster's operator would contain the coarse neighbour's face stress without the volume term that keeps it positive. Splitting by element owner keeps every part positive semidefinite, provided each side's penalty dominates (§7.3).

Step 2: split the operator by element owner. Write the code's face form (§7.3) as two side halves, $a_F = a_F^{(1)} + a_F^{(2)}$, with

$$a_F^{(i)}(u,w) = -\tfrac{1}{2}\int_F \sigma(u_i)n\cdot[\![w]\!] - \tfrac{1}{2}\int_F \sigma(w_i)n\cdot[\![u]\!] + \pi\,\tfrac{p_i}{4}\int_F [\![u]\!]\cdot[\![w]\!],$$

where π is the deck's `penalty_factor` and p_i the side penalty of `ComputePenalty`. Let $G_e = \mathrm{vol}_e + \sum_{F\ni e} a_F^{(e)}$, with whole boundary faces. Then $K = \sum_e G_e$ exactly, and cluster c owns $K^{(c)} = \sum_{e\in c} G_e$; K_η splits the same way. Both tets of a fault face are in one cluster, so the slip loads and fault face terms all belong to the fault cluster. Each G_e annihilates rigid translations, so every kick conserves linear momentum (inference).

Step 3: the loop over one sync interval $[t_S, t_S + \Delta t_L]$, with $L = N_c - 1$ (derived in §7.1):

```
for m in 0 .. 2^L-1:                          # tick m: t_m = t_S + m dt0
    for c starting a step at t_m:   v[rows of K^(c)] += dt_c/2 a[c]
    u += dt0 v;  t = t_m + dt0                # drift (lazily: only DOFs that are read)
    E = { c : (m+1) % 2^c == 0 }              # clusters ending a step at t
    if fault cluster cF in E:  AdvanceFault_(u, v, t) with dt = dt_cF
    exchange the ghosts that the clusters in E read      # one merged round per tick
    for c in E:                               # every force sees the same (u, v-)
        r = F_D^(c)(t) - K^(c) u - K_eta^(c) v + [c == cF](G d + G_eta V) - C^(c) v
        a[c] = (M + dt_c/2 C)^(-1) r on the rows of K^(c)   # per-element dt_c
    for c in E:   v[rows of K^(c)] += dt_c/2 a[c]
# at t_S + dt_L all velocities are synchronised: output, energy, step record
```

With one cluster this is today's step in kick–drift–kick form (§7.1). The order of operations differs from `NewmarkSolver::Step`, so $N_c = 1$ agrees with current runs to round-off, not bit for bit. Runs with `lts = "off"` keep `CentralDifferenceSolver` and stay bit-identical.

Step 4: fault. `AdvanceFault_` runs unchanged at the fault cluster's instants with $\Delta t_F = \Delta t_0 2^{c_F}$. All fault-adjacent DOFs are in c_F, so $u(t)$ there is the physical drifted displacement and v^-

plays the role of v^*. The `MPI_Allreduce` of the step record (:909) becomes a running maximum reduced once per sync interval.

Step 5: damping. Each cluster lags its damping by half of its own step, as today. With `kv_scale = "node"`, $\beta_e \propto h_e/p$, so $r = \frac{1}{2}\beta_e\omega_{\max,e}$ and the damped fraction $\sqrt{1+r^2} - r$ do not depend on element size (KV derivation rev 2.1, §5), and `cfl = 0.5` keeps the same margin in every cluster. This is an inference; seam coupling could lower it, and P2 measures it. With `kv_scale = "dt"` there is no single Δt, so LTS should refuse it.

Step 6: absorbing faces and Dirichlet data. `NitscheMass` takes one `half_dt` (nitsche_mass.hpp:53-57); it needs $\frac{1}{2}\Delta t_{c_e}$ per element. Dirichlet data is assembled per cluster at that cluster's instants.

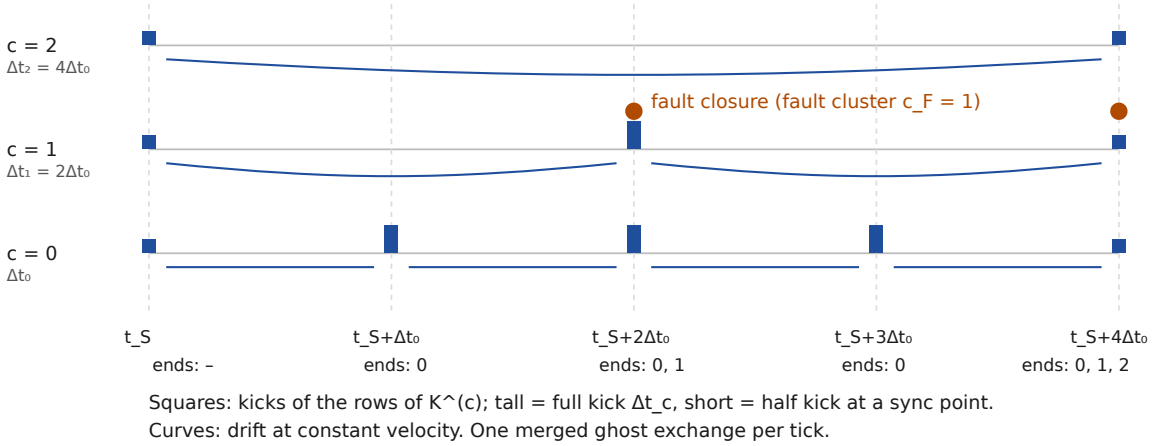


Figure 2. One sync interval with three clusters. Each cluster kicks at its own instants and drifts in between; the fault closes at its cluster's instants, here $c_F = 1$ as on both 200 m meshes. A cluster-1 DOF on a seam is also kicked every Δt_0 by the G_e of its cluster-0 neighbour.

3.5 Code structure

The velocity-stress (ADER) path already holds a clustered-LTS stack in `miniapps/seas/dynamic/`. The parts that do not depend on the time scheme carry over.

Existing piece (link)	What it does	Reuse here
BuildLtsClustering	Rate-2 binning with SeisSol's rules, wiggle scan, cap, CFL assert.	**As is,** with the Nitsche Δt_e.
BuildLtsAwarePartition, … Imbalance	Multi-constraint METIS; fault faces on one rank; imbalance report.	**As is,** plus item 6's fault weight; feeds `ParMesh` at driver:759.
BuildTickTable	Clusters starting and ending at each tick.	**Tick arithmetic only.**
BuildLtsLayout, LtsLayoutHash	Cluster element and face lists (ADER roles).	**Structure only;** needs half-face lists.
`lts_bulk_stepper.hpp`, `lts_fault_stepper.hpp`, `lts_time_basis.hpp`	ADER predictor–corrector and flux buffers.	**Not reusable:** a leapfrog DOF at any instant is $u + \tau v$.

The deck key `[numerics] lts` belongs to the velocity-stress path, and the Nitsche driver lists it among ignored keys (driver:740-746). Item 7 gives it a Nitsche meaning (`"off"` or `"rate2"`, plus a cluster cap). The new pieces are these.

- **`NitscheLtsSetup`** (driver, before `ParMesh`). Inputs: serial mesh, material, p, `cfl`, fault attribute. Outputs: cluster ids, Δt_0, partition, and a `--lts-report` line. It works on the serial mesh, because cluster ids must be a pure function of it (lts_clustering.hpp:12-15).
- **`NitscheLtsLayout`.** Per cluster: its elements, its half-faces (F, side), the rows $K^{(c)}$ kicks, and the ghosts it reads.
- **Per-cluster operators.** Either (i) assembled, with a side-masked copy of the combined integrator that keeps side i's consistency, symmetry and $\pi p_i/4$ penalty terms, stored on the cluster's own rows with seam blocks on the trace pattern; or (ii) item 8's kernel looping over the cluster's elements and half-faces. Route (ii) is the natural one, which is why item 8 comes first.
- **`NitscheLtsStepper`.** Replaces `CentralDifferenceSolver` at driver:1431-1444; `Step` advances one sync interval with the loop of §3.4, calling a cluster-restricted `Acceleration_` and the unchanged `AdvanceFault_`.
- **Output.** Stations at fault-cluster instants or sync points, with a time interval in place of `--station-every`; fault frames, energy and log at sync points.

Parallel. The velocity-stress path shows the hazard. In its LTS runs on TPV104 200 m, exposed `MPI_Waitall` was 40 to 53% of the wall time, and halving the exchange rounds did not reduce it, because late senders caused it (`document/kernel_dev/ANALYSIS_seissol_mechanism_gap_2026-07-24.md`, M4; `document/comm_dev/PLAN_lts_comm_reduction_2026-07-22.md`). The Nitsche design therefore uses one merged exchange per tick, posted early and completed late; a multi-constraint partition, so every cluster is balanced on every rank; and a fault weight. At $p = 1$ a tet-step costs 3.5 to 5.4 µs per core on Vista (profiling §4.1), and a cluster-0 tick on 576 ranks has about 14 cluster-0 tets per rank on TPV205 200 m. Its compute, under 0.1 ms, is comparable to one exchange's latency, so the cluster cap and the exchange design matter more here than on the $p = 3$ velocity-stress path (inference).

3.6 Risks

- **Stability.** No energy proof exists for family (B), and resonance bands or seam coupling could lower the damped limit. P1 (c) and P2 (a) in §3.7 measure both before any physics run.
- **Positivity on slivers.** The penalty scales with $|F|/|K|$, so slivers have the stiffest face terms, and P1 (b) is tightest there. If it fails, the fix changes the penalty, which changes results and needs Chunhui Zhao's decision.
- **Friction at a larger step.** With the fault in cluster 1, Δt_F is twice today's step, which the check $|V|\Delta t/D_c \le 2$ (:887) and γ see. If the check fails, the fault goes into cluster 0 (4.26× instead of 4.54× ideal).
- **Item 2 and new golds.** Optimised meshes remove much of the gain and move the fault into cluster 0, so item 2 should be decided before P4. Results change by the time-discretisation error, so every benchmark needs a new gold.

3.7 Phases, tests and expected gain

Each phase ends with a check that gates the next. Every cluster run follows the repository's GitHub-first rule: code, decks and job scripts are pushed first, the job runs the commit guard, and the result records the commit.

Phase	Content	Acceptance test	Gain at the end
P0	`--lts-report`: clustering on the Nitsche Δt_e, histograms, fault faces per cluster, partition imbalance.	Numbers equal the §7.6 script on the four meshes of §3.1. Decide the cluster cap and the mesh (item 2).	None (information).
P1	Serial multirate leapfrog, pure elastic; two clusters, then N_c.	(a) $N_c = 1$ equals `CentralDifferenceSolver` to round-off. (b) Every G_e is positive semidefinite (smallest eigenvalue $\geq -10^{-12}$ of the largest, §7.3). (c) Spectral radius of the sync-interval map (power iteration) at most $1 + 10^{-10}$ for $\Delta t_0/\Delta t_{\mathrm{CFL}} = 0.30, 0.31, \ldots, 1.00$, to catch resonance bands. (d) Energy at sync points bounded over 10^5 intervals. (e) Second-order convergence in time.	Not timed yet; 1000 m block count 3.23× at $p = 1$.
P2	Kelvin–Voigt per cluster; absorbing faces with per-element $\frac{1}{2}\Delta t_c$.	(a) Stable fractions per cluster match KV rev 2.1 §6 (0.78, 0.61, 0.41 at $\bar{\eta} = 0.05$, 0.1, 0.2). (b) Damping power ≤ 0. (c) Absorbing reflection as in the global run.	As P1.

Phase	Content	Acceptance test	Gain at the end
P3	Fault in one cluster; `AdvanceFault_` at Δt_F; stations at fault instants.	(a) $N_c = 1$ reproduces a TPV205 2000 m run to round-off. (b) All clusters forced to Δt_F equals a global run at Δt_F. (c) TPV205 1000 m with LTS differs from global stepping by no more than the global run changes from Δt to $\Delta t/2$. (d) Energy balance with friction work closes.	Serial 1000 m: at most 3.76×, 3.23× by block count.
P4	Parallel: LTS partition, merged and overlapped exchange per tick, fault weights, record reduced per sync.	(a) np = 1, 4, 16 agree to round-off. (b) Exchange count equals the tick table (as the velocity-stress P-007 assert does). (c) Rank sweep reports per-cluster imbalance and exposed wait.	Unknown; 50 to 80% of the block count is a guess.
P5	TPV205 and TPV104, 200 m, $p = 1, 2$, on Vista.	Rupture time, peak slip rate and final slip agree with the SCEC references as well as the current golds do. New golds record their commits.	Step compute on TPV205 200 m $p = 1$ is 742 s of 1102 s (§5); at 2 to 3× it saves 370 to 495 s (inference).

4. Item 8: custom SIPG kernels

4.1 Why the products are slow

The CSR matrix holds 707 entries per tet at $p = 1$ and 4324 at $p = 2$, which is 8.5 kB and 52 kB per tet and matrix (profiling §4.3). A product does 2 flops per entry and reads 6 bytes per flop, so it runs at memory speed: 47% of stream bandwidth at $p = 1$ and 58 to 66% at $p = 2$, or 2.0 and 7.7 µs per tet and product on one core (127 ms on 62.6k tets and 86 ms on 11.2k tets, §4.3). About 80% of the entries are face couplings, which MFEM cannot apply matrix-free on tets: partial assembly of the volume part gained nothing (0.86 to 1.03 times), and dense volume blocks gave 1.13 to 1.44 times (§4.3). `DGElasticityIntegrator` has only `AssembleFaceMatrix` (fem/bilininteg.hpp:3691-3702), and the code's combined integrator has no partial or element assembly either.

4.2 Options, from simplest to fastest

Each option computes the same $Ku + K_\eta v$. Four points stand behind the table that follows, whose figures are for both operators per tet (§7.5 derives them).

- **The byte model is checked.** It predicts 1.21 times for item 4, and the benchmark measured 1.10 to 1.26 times.
- **Symmetry.** K is exactly symmetric (:183-194), so storing one off-diagonal block per face and half of each diagonal block nearly halves the bytes again.
- **Few flops in (b) at $p = 1$.** On an affine tet the gradients are constant and the face integrals reduce to reference face masses, so (b) needs 0.37 times the flops of CSR.
- **The open question for (b) is its flop rate.** The velocity-stress path's generic `DenseMatrix` and AXPY code ran at about 0.4 GFLOP/s per core against SeisSol's 6.4 (`ANALYSIS_seissol_mechanism_gap_2026-07-24.md`, M1), and SeisSol's rate rises from 2.62 to 7.84 GFLOP/s from order 2 to 4 (`RESULTS_tpv104_gts_order_baseline_2026-07-25.md`). Kernels built from `DenseMatrix` calls could be slower than CSR at $p = 2$; they need fixed-size loops templated on p, or generated code.

Option	Bytes per tet, $p = 1$ / $p = 2$	Flops per tet, $p = 1$ / $p = 2$	Speed of the two products
Today: two CSR products	17.4 / 105 kB	2.8 k / 17.3 k	Measured 4.1 / 15.4 µs per tet (1.0×)
Item 4: fused CSR	14.4 / 87 kB	same	1.21× predicted; 1.10–1.26× measured
(a1) dense blocks, fused	11.6 / 70 kB	same	About 1.5× (bytes)
(a2) symmetric blocks, fused	6.2 / 37 kB	same	2.8× by bytes; **2.1–2.7×** at the measured 75–96% dense-block efficiency
(b) matrix-free, fused	0.6–1.3 / 1.0–2.1 kB	1.0 k / 13–18 k	Set by the flop rate: at 1 to 5 GFLOP/s, 4–16× ($p = 1$), 0.9–6× ($p = 2$)
(c) (b) with generated GEMMs	as (b)	as (b)	Upper end of (b)

With the products at 87% of a step, a speedup k on them gives $1/(0.13 + 0.87/k)$ per step: 1.8 times at $k = 2.1$, 2.2 at $k = 2.7$, 3.6 at $k = 6$, and at most 7.7.

4.3 How the SIPG operator decomposes on affine tets

The basis is nodal Gauss–Lobatto, ordered by nodes (elasticity_operator_setup.inl:11-13). On a face only the face nodes are non-zero, 3 at $p = 1$ and 6 at $p = 2$, which nodal DG codes exploit (Hesthaven and Warburton 2008).

- **Volume:** $\int_e \sigma(u) : \varepsilon(w)$ with J_e, λ_e, μ_e constant. At $p = 1$, ∇u is constant; at $p = 2$ a 4-point degree-2 rule is exact.
- **Interior faces, fault faces included:** $a_F(u, w) = -\int_F \{\{\sigma(u)n\}\} \cdot [\![w]\!] - \int_F \{\{\sigma(w)n\}\} \cdot [\![u]\!] + \int_F \sigma_F [\![u]\!] \cdot [\![w]\!]$, with $\sigma_F = \pi(p_1 + p_2)/4$ and $p_i = 4\,\frac{p(p+2)}{3}\,\frac{|F|}{|K_i|}\,\frac{(3\lambda_i+2\mu_i)^2}{2\mu_i}$ (§7.3; ComputePenalty :857-883). On an affine face σ_F is one number. The integrands have degree at most $2p$, so any rule exact to that degree reproduces the assembled matrix to round-off.
- **Dirichlet faces:** one side, $c_0 = -1$, $c_1 = \epsilon = -1$, penalty πp_1 (:123-141, :872-875). Absorbing and free faces add nothing to K; the dashpot stays in its element blocks (:1155).
- **Shared faces:** each rank computes its own side's rows from the neighbour's ghost values, exactly as `KeepNbrBlock(true)` does; the `v57` comment explains why this keeps results partition-independent (assembly.inl:25-34).

K_η in the same pass. K_η uses the same integrator with moduli $\beta_e\lambda_e, \beta_e\mu_e$ (:636-660), and the penalty is linear in β, so $\sigma_F^\eta = \pi(\beta_1 p_1 + \beta_2 p_2)/4$. Because β_e is constant per element, the volume and consistency terms of $Ku + K_\eta v$ are those of K applied to $z = u + \beta_e v$ on each side; only the symmetry and penalty terms need $[\![u]\!]$ and $[\![v]\!]$ separately (§7.4). The fused kernel therefore costs about one operator plus the extra jumps (inference). This does not need $K_\eta = \beta K$, which is false because β_e varies about 100 times across elements (profiling §5.4).

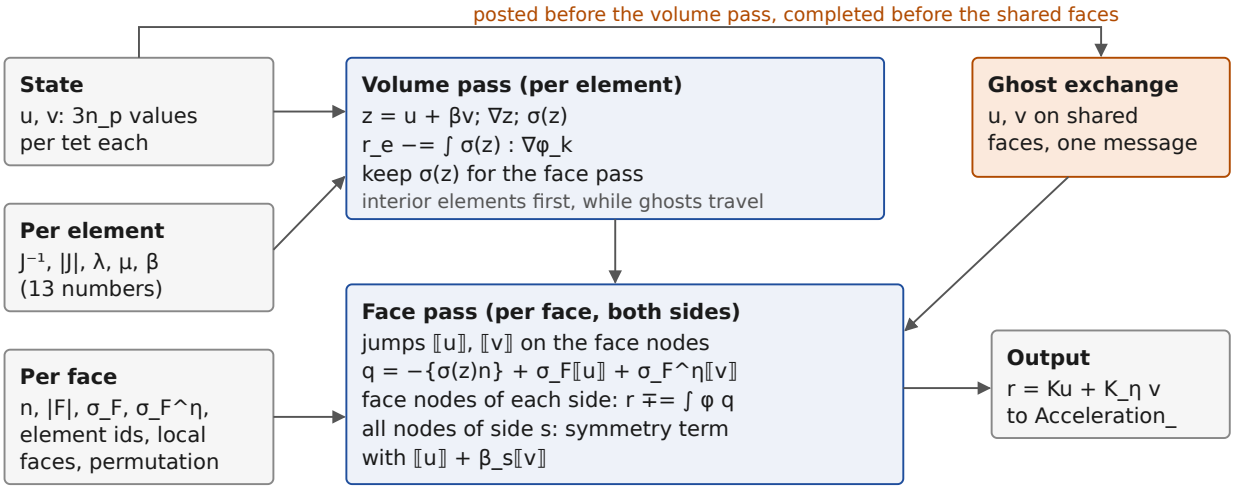


Figure 3. Data flow of the fused matrix-free kernel (option b). Option (a2) has the same two passes with stored blocks in place of the formulas. Under LTS both passes run over one cluster's elements and half-faces.

4.4 Code structure

- **`NitscheSipgOperator`**, an `mfem::Operator` built from the same material sample, penalty factor and boundary markers as `ElasticityDomainOperator`. Setup inputs: the `ParMesh`, the DG space, $\lambda_e, \mu_e, \beta_e$. Data: element geometry; a face list (interior, shared, Dirichlet) with element ids, local face numbers and vertex permutations; for (a2), the symmetric blocks read out of the assembled matrix's `diag` and `offd` parts through the element DOF lists. Methods: `Mult(u, r)` and `MultFused(u, v, r)`, which returns $Ku + K_\eta v$.
- **Ghosts:** one exchange of u and v through the face-neighbour tables of `ParFiniteElementSpace`, replacing the two internal exchanges of the two `HypreParMatrix::Mult` calls.
- **Call site:** in `Acceleration_` the two `ApplyStiffness` calls (:786, :803) become one `MultFused` behind a deck switch. The assembled matrices stay as the test reference; with item 5 they are freed after setup.

```
r = MultFused(u, v):                                  # option (b); (a2) applies stored blocks
    post the ghost exchange of (u, v)
    for e in elements:                                # volume pass
        z = u_e + beta_e v_e;  G = grad z (via J_e^-1);  S_e = sigma(G; lam_e, mu_e)
        r_e -= |e| S_e : grad phi_k                   # quadrature at p = 2
    wait for ghosts
    for F in faces (interior, shared, Dirichlet):  # face pass
        ju, jv = jumps of u, v at the face nodes
        q = -avg(S n) + sigma_F ju + sigma_F^eta jv
        r(face nodes, side 1) -= int phi q;  r(face nodes, side 2) += int phi q
        for each side s: r_s += symmetry term of (ju + beta_s jv)
    return r                                          # = K u + K_eta v to round-off
```

4.5 Risks and effort

- **Flop rate of (b)** is the main unknown; P0 measures it before the solver is touched.
- **Round-off growth.** A nonlinear rupture run can amplify last-bit differences; P5 compares against the golds.
- **Memory** improves: (a2) needs 2.8 times less than the two CSR matrices and (b) almost none, which helps the $p = 2$ runs whose setup peak is about 300 kB per tet (profiling §4.7).
- **Effort.** (a2) is medium, since the blocks come from the assembled matrix. (b) is large: two passes, three face kinds, orientations, ghosts and two orders.

4.6 Phases and tests

Every phase compares the new product with the assembled one, $\|r_{\text{new}} - r_{\text{asm}}\|_\infty \le 10^{-13}\|r_{\text{asm}}\|_\infty$, so results stay as they are. P0 extends the existing benchmark `miniapps/seas/tests/unit/bench_nitsche_kernels.cpp` (profiling §6), which is on GitHub as commit `b5427c8` of branch `claude/nitsche-profiling-w336yq`. The Vista run in P5 follows the GitHub-first rule like any cluster run.

Phase	Content	Acceptance test (against the assembled product)	Gain at the end
P0	Benchmark prototypes of (a2), and of (b) at $p = 1$.	Both match; time per tet decides (a2) alone or (a2) then (b).	Measured in the benchmark.
P1	Serial volume kernel.	Volume-only matrix, $p = 1, 2$.	—
P2	Interior faces, fault faces included.	Full K without Dirichlet faces, random u.	—
P3	Dirichlet faces; call-site switch, serial.	K of the TPV205 and TPV104 decks; 200 steps of TPV205 2000 m match the stations to 10^{-10}.	Serial step 1.8–2.2× with (a2) (inferred).
P4	Parallel: shared faces, one ghost exchange, overlap.	np = 1, 4, 16 give the same r.	As P3 per core.
P5	K_η fused (`MultFused`).	Both products; a Vista TPV205 200 m $p = 1$ run matches the gold stations to round-off growth.	Full gain of the chosen option.
P6	Optional: OpenMP with face colouring, generated kernels for $p \geq 2$, GPU.	As P5.	Not estimated.

5. How items 7 and 8 combine, and what to do first

Item 8 lowers the cost of each element update and item 7 lowers the number of updates, so they multiply on the step compute but not on setup, output or the fault. On TPV205 200 m $p = 1$ the step compute is 742 s of the 1102 s wall (1102 s less 68 s setup and 292 s fault output, profiling §4.1). Option (a2) at 1.8 to 2.2 times per step and LTS at a realised 2 to 3 times would take it to about 110 to 210 s (inference). Output would then dominate unless item 1 is done first, so the profiling plan's order still leads.

1. **Items 1, 5, 3, 4 and 6** as planned; item 4's fused CSR loop is the first step of 8a.
2. **Item 2 (mesh quality)**, because it changes the golds and the LTS histogram (§3.1).
3. **8a**, P0 to P5, with results unchanged.
4. **8b**, only if P0 shows it beats 8a at the orders in use.
5. **Item 7**: P0 at any time, then P1 to P5 on item 8's loops; P5 needs new golds.

6. Decisions needed from Chunhui Zhao

- Whether item 8 may change last bits, with the golds kept and checked in P5 rather than rerun.
- Family (B) for LTS, the P1 gates, and the cluster cap (four clusters give 93 to 96% of the ideal).
- Whether item 2 comes before item 7 P4, and whether LTS may refuse `kv_scale = "dt"`.

7. Derivations and supporting detail

7.1 From today's step to the multirate loop

Assumptions: (A1) one cluster with step Δt; (A2) $a = M^{-1} f(u, v)$, with v entering f only through damping and absorbing terms.

Step 1 (ode.cpp:1271-1304, $\beta = 0$, $\gamma = \frac{1}{2}$). Newmark gives

$$u^{n+1} = u^n + \Delta t\, v^n + \tfrac{1}{2}\Delta t^2 a^n, \quad v^* = v^n + \tfrac{1}{2}\Delta t\, a^n, \quad a^{n+1} = M^{-1} f(u^{n+1}, v^*), \quad v^{n+1} = v^* + \tfrac{1}{2}\Delta t\, a^{n+1}.$$

Step 2 (algebra). With $v^{n+1/2} := v^*$, $u^{n+1} = u^n + \Delta t\, v^{n+1/2}$, which is kick–drift–kick. Since $v^{n+3/2} = v^{n+1} + \frac{1}{2}\Delta t\, a^{n+1}$, the end kick of step n and the start kick of step $n+1$ share a^{n+1}, evaluated once at $(u^{n+1}, v^{n+1/2})$.

Step 3 (split $f = \sum_c f^{(c)}$, §3.4 step 2). Give each $f^{(c)}$ its instants $t_S + k\Delta t_c$ and let every DOF drift between kicks. At an instant all due parts use the same $(u(t), v^-(t))$, with v^- the velocity before that instant's kicks, so the kick order does not matter. With one cluster $v^-(t_{n+1}) = v^{n+1/2}$ and Step 2 is recovered. This is the nested impulse method (Tuckerman et al. 1992) in kick–drift–kick form.

Step 4 (fault, AdvanceFault_). Today $d^* = d^n + \frac{1}{2}\Delta t V^n$, $d^{n+1} = d^* + \frac{1}{2}\Delta t V^{n+1}$, and the end-of-step traction is $X^* + \gamma V^{n+1}$ with $\gamma = \frac{1}{2}\sigma_F \Delta t + \sigma_F^\eta$. Only fault DOFs and the fault cluster's step appear, so replacing Δt by Δt_F gives the fault step of the multirate loop.

7.2 The leapfrog LTS of Diaz and Grote, and why its substeps are not physical

For $z'' + Az = R$, with P the refined-region selector and p substeps (Grote and Mitkova, arXiv:1205.0654, LTS-LF2(p)):

$$w = (I - P)R_n - A(I - P)z_n, \quad q_0 = 2z_n, \quad q_{1/p} = q_0 + \tfrac{1}{2}\big(\tfrac{\Delta t}{p}\big)^2\big(2w + 2PR_{n,0} - APq_0\big),$$

$$q_{\frac{m+1}{p}} = 2q_{\frac{m}{p}} - q_{\frac{m-1}{p}} + \big(\tfrac{\Delta t}{p}\big)^2\big(2w + P(R_{n,m} + R_{n,-m}) - APq_{\frac{m}{p}}\big), \qquad z_{n+1} = -z_{n-1} + q_1.$$

Set $q(\tau) := z(t_n + \tau) + z(t_n - \tau)$. Then $q(0) = 2z_n$, $q'(0) = 0$, $q'' = -A\,q + R(t_n + \tau) + R(t_n - \tau)$, and $q(\Delta t) = z_{n+1} + z_{n-1}$. The scheme freezes the coarse part of Aq at $2A(I - P)z_n$ and integrates the rest with leapfrog. So $q_{m/p}$ approximates $z(t_n + m\Delta\tau) + z(t_n - m\Delta\tau)$, not $z(t_n + m\Delta\tau)$, and sources are needed at $t_n \pm m\Delta\tau$. ∎

7.3 The code's face form, its split by side, and positivity

Step 1 (:123-181). The normal points out of element 1 and $[\![u]\!] = u_1 - u_2$. Block (1,1) has $c_0 = -\frac{1}{2}$, $c_1 = \frac{1}{2}\epsilon = -\frac{1}{2}$ and $c_2 = +\sigma_F$; block (1,2) has $-\frac{1}{2}$, $+\frac{1}{2}$ and $-\sigma_F$, the same form with $[\![u]\!] = -u_2$. So

$$a_F(u, w) = -\int_F \{\{\sigma(u)n\}\} \cdot [\![w]\!] - \int_F \{\{\sigma(w)n\}\} \cdot [\![u]\!] + \int_F \sigma_F [\![u]\!] \cdot [\![w]\!], \qquad \sigma_F = \pi\, \frac{p_1 + p_2}{4}.$$

Step 2 (`ComputePenalty`, :857-883, with $D = 3$, $\mathtt{nl} = 2|F|$, $\det J_i = 6|K_i|$). Since $(D + 1)\, c_N\, D\, \mathtt{nl} / \det J_i = 4 \cdot \frac{p(p+2)}{3} \cdot \frac{|F|}{|K_i|}$, we get $p_i = 4\, \frac{p(p+2)}{3}\, \frac{|F|}{|K_i|}\, \frac{(3\lambda_i + 2\mu_i)^2}{2\mu_i}$.

Step 3 (split). $\{\{\sigma n\}\} = \frac{1}{2}\sigma_1 n + \frac{1}{2}\sigma_2 n$ and $\sigma_F = \pi p_1/4 + \pi p_2/4$ give $a_F = a_F^{(1)} + a_F^{(2)}$ as in §3.4, hence $K = \sum_e G_e$.

Step 4 (positivity; an inference that P1 (b) checks). $G_e(u, u) = \|\varepsilon(u_e)\|_{\mathbb{C}}^2 - \sum_{F \ni e} \int_F \sigma(u_e) n \cdot [\![u]\!] + \sum_F \pi \frac{p_e}{4} \|[\![u]\!]\|_F^2$. Young's inequality and the discrete trace inequality on element e alone bound the middle term by part of $\|\varepsilon(u_e)\|_{\mathbb{C}}^2$ plus a multiple of $\frac{|F|}{|K_e|}\frac{(3\lambda + 2\mu)^2}{2\mu}\|[\![u]\!]\|_F^2$, as in the per-element

step of the standard SIPG coercivity proof. So $G_e \geq 0$ once $\pi p_e/4$ exceeds that multiple. Whether the factor $\frac{1}{4}$ is enough for every element is not proven here; the test computes the smallest eigenvalue of each G_e on its patch (60 DOFs at $p = 1$, 150 at $p = 2$).

7.4 The fused form of $Ku + K_\eta v$

Per element β_e is constant (:636-638). Each term, labelled by source, and what the fused kernel computes:

Term	From K (on u)	From K_η (on v)	Fused
Volume	$\int_e \sigma(u) : \varepsilon(w)$	$\int_e \beta_e \sigma(v) : \varepsilon(w)$	$\int_e \sigma(z) : \varepsilon(w)$, $z = u + \beta_e v$
Consistency	$-\{\{\sigma(u)n\}\} \cdot [\![w]\!]$	$-\{\{\beta\sigma(v)n\}\} \cdot [\![w]\!]$	$-\{\{\sigma(z)n\}\} \cdot [\![w]\!]$
Symmetry, side s	$-\frac{1}{2}\sigma(w_s)n \cdot [\![u]\!]$	$-\frac{1}{2}\beta_s\sigma(w_s)n \cdot [\![v]\!]$	$-\frac{1}{2}\sigma(w_s)n \cdot ([\![u]\!] + \beta_s[\![v]\!])$
Penalty	$\sigma_F [\![u]\!] \cdot [\![w]\!]$	$\sigma_F^\eta [\![v]\!] \cdot [\![w]\!]$	$(\sigma_F [\![u]\!] + \sigma_F^\eta [\![v]\!]) \cdot [\![w]\!]$

7.5 Bytes and flops per tet

Assumptions: $n_p = 4$ nodes at $p = 1$ and 10 at $p = 2$, $3n_p$ DOFs, about two interior faces per tet, 8-byte reals, 4-byte indices, vectors read or written once.

- **CSR** (measured nnz): $12 \cdot 707 = 8.5$ kB and $12 \cdot 4324 = 51.9$ kB per matrix; two products with vectors, 17.4 and 104.8 kB; $2 \cdot 2 \cdot \text{nnz}$ flops, 2.8 k and 17.3 k.
- **Fused CSR**: $20 \cdot \text{nnz}$ plus vectors, 14.4 and 87.2 kB (ratio 1.21). **(a1)**: $16 \cdot \text{nnz}$ plus vectors, 11.6 and 70 kB.
- **(a2)**: per matrix and tet, half a diagonal block, $n(n+1)/2$ with $n = 3n_p$ (78 and 465), plus two off-diagonal blocks of n^2 (288 and 1800): 2.9 and 18.1 kB; for two matrices plus vectors, 6.2 and 36.9 kB.
- **(b) bytes**: 13 element numbers (104 B), about 60 B per face twice, vectors (288 and 720 B), plus 576 and 1152 B if neighbour face values come from memory: 0.6–1.3 and 1.0–2.1 kB.
- **(b) flops,** $p = 1$: volume about 190 (z 24, ∇z 72, stress 15, residual 81); face about 430 (jumps 18, integrals 24, flux 39, consistency 36, penalty 108, symmetry 200); total $190 + 2 \cdot 430 \approx 1.05$ k.
- **(b) flops,** $p = 2$: volume 4 points at about 490, 2.0 k; face 6 points at about 900 to 1300, 5.4 to 7.8 k per face; total 13 to 18 k.
- **Time** is the larger of bytes over bandwidth (9 to 12 GB/s per core, inferred from the measured 47–66% shares) and flops over 1 to 5 GFLOP/s: 0.25 to 1.1 µs against 4.1 µs at $p = 1$, and 2.6 to 18 µs against 15.4 µs at $p = 2$.

7.6 The mesh-statistics scripts

`lts_design_stats.py` (inputs: a msh 2.2 mesh and the fault tag, 103 for TPV205 and 3 for TPV104) computes $h_e = 6V/A$, face adjacency and fault pairs. It applies the rules of `lts_clustering.cpp` (integer binning, max-difference fixpoint, fault rule, λ scan over (0.5, 1], caps), finds element layers from the fault by breadth-first search, and evaluates the two-level Diaz–Grote cost $2/(1 + 2f)$, with f the share of rows touched by AP, for overlaps 0 to 2. It assumes homogeneous material, which holds for both benchmarks.

`lts_seam_sums.py` computes the block count of §3.1. Each element's diagonal block and each intra-cluster face's two off-diagonal blocks are charged at that cluster's rate. Each side of a seam face is charged, at its own rate, for the two rectangular blocks of its half-form (the neighbour's face nodes against its own nodes, and the transpose) and the neighbour's face-mass block from the penalty. Global stepping costs one diagonal block per element and two off-diagonal blocks per face every Δt_0. In the full model every block is dense $3n_p \times 3n_p$, as today's CSR stores it (`Assemble(0)` keeps the zeros; 707 entries per tet is close to $5 \cdot 144 = 720$). In the trace model, an off-diagonal block of K has $(3n_p)^2 - (3(n_p - n_f))^2$ entries with n_f face nodes (135 at $p = 1$, 756 at $p = 2$), and a seam side has $2 \cdot 3n_f \cdot 3n_p + 3n_f^2$ (243 and 1188). In the full model the split operators store 37% more blocks than K on the 200 m meshes.

8. References

Verified in this session (abstract or record read): Diaz, J. & Grote, M. J. (2009), SIAM J. Sci. Comput. 31(3):1985–2014 (record via the research notes). Grote, M. J. & Mitkova, T., "Explicit local time-stepping methods for time-dependent wave propagation", arXiv:1205.0654 (LTS-LF2(p), LTS-LFCN, the overlap and 60% results). Grote, M. J. & Mitkova, T. (2013), J. Comput. Appl. Math. 239:270–289, arXiv:1109.4480. Grote, M. J., Michel, S. & Sauter, S., "Stabilized leapfrog based local time-stepping method for the wave equation", arXiv:2005.13350 (Math. Comp. 2021). Grote, M. J., Mehlin, M. & Sauter, S., arXiv:1703.07965. Rietmann, M., Grote, M., Peter, D. & Schenk, O. (2017), J. Comput. Phys. 334:308–326. Rietmann, M. et al. (2015), "Load-balanced local time stepping for large-scale wave propagation", IPDPS. Lew, A., Marsden, J. E., Ortiz, M. & West, M. (2003), "Asynchronous variational integrators", Arch. Rational Mech. Anal. 167:85–146. Tuckerman, M., Berne, B. J. & Martyna, G. J. (1992), J. Chem. Phys. 97:1990. García-Archilla, B., Sanz-Serna, J. M. & Skeel, R. D. (1998), "Long-time-step methods for oscillatory differential equations", SIAM J. Sci. Comput. 20 (title only). SeisSol documentation, "Local time-stepping". Uphoff, C. & Bader, M. (2020), YATeTo, ACM TOMS, arXiv:1903.11521. Uphoff, C., May, D. A. & Gabriel, A.-A. (2023), GJI 233(1):586–626. Breuer, A., Heinecke, A. & Bader, M. (2016), IPDPS. Uphoff, C. et al. (2017), SC17. Krenz, L. et al. (2021), SC21.

From memory, not verified here: Biesiadecki, J. J. & Skeel, R. D. (1993), "Dangers of multiple time step methods", J. Comput. Phys. 109. Hesthaven, J. S. & Warburton, T. (2008), Nodal Discontinuous Galerkin Methods, Springer.

9. Where it lives

- This document: `/mnt/project-files/profiling/lts-kernels-design-rev1.pdf`, with `lts-kernels-design-rev1-source.html` beside it. Code was read at `53ba400` (branch `nitsche-benchmarks`) from GitHub; no repository file was changed, built or run.
- Repository documents: `document/lts_dev/ANALYSIS_lts_method_selection_2026-07-18.md`, `document/comm_dev/PLAN_lts_comm_reduction_2026-07-22.md`, `document/kernel_dev/ANALYSIS_seissol_mechanism_gap_2026-07-24.md`, `document/kernel_dev/RESULTS_tpv104_gts_order_baseline_2026-07-25.md`, `jobs/lts_phase5/tpv104_200m_seissol_expanse/README.md`.
- Profiling report: `/mnt/project-files/profiling/nitsche-profiling-plan-rev1.pdf`. KV derivation: `/mnt/project-files/damping/kv-damping-derivation-rev2.1.pdf`.
- Meshes: `tpv205_200m.msh` and `tpv104_200m.msh`, regenerated from the repository geometries in this session's scratchpad (not the gold meshes themselves); the repository's `tpv205/mesh/tpv2053d_1000m.msh`; and `w_d.msh`, the same geometry with `Mesh.Algorithm3D = 10; OptimizeThreshold = 0.6; OptimizeNetgen = 1`. Scripts `lts_design_stats.py` and `lts_seam_sums.py` are in the session scratchpad.
- Tandem's source is not in this container; statements about Tandem rest on its GJI paper and the integrator's comments (:1-12).